PCCCS495: ADVANCED PROGRAMMING (OOP) LAB

MINI BANKING SYSTEM

NAME – PRATYUSHA NANDI

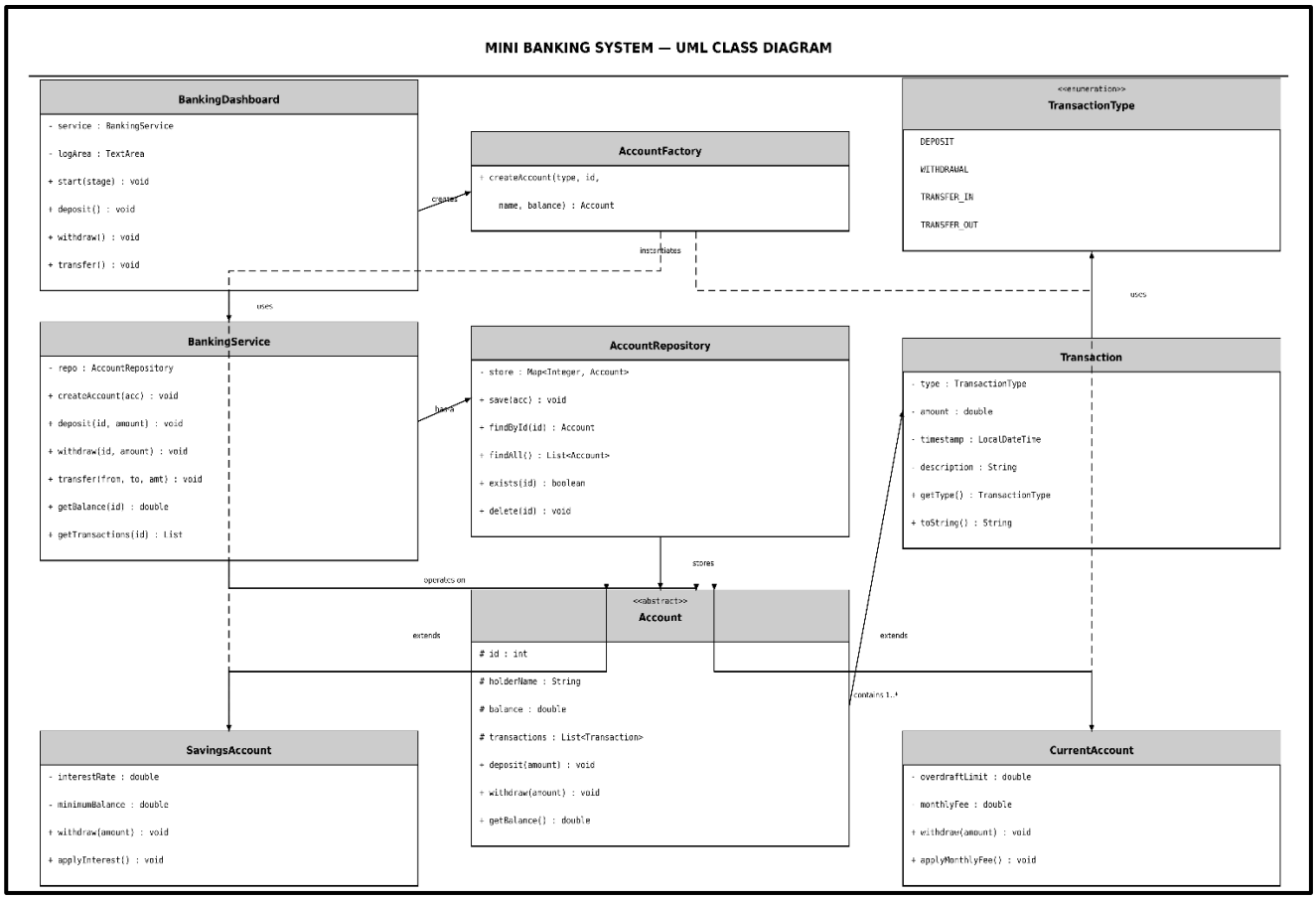
ROLL NO – 67 (CSE IOTCSBT)

Instructors: Prof Swagatam Basu, Prof Susovan Jana, Prof Soumadip Biswas

SPRING 2026

Problem Statement: **Mini Banking System using Layered Architecture**

SYSTEM DESIGN



MAJOR CLASSES:

Account (Abstract Class) – Base class containing common fields (ID, name, balance, transactions) and defining core operations like deposit and abstract withdraw.

SavingsAccount – Basic account type allowing withdrawal only when sufficient balance is available.

CurrentAccount – Account type supporting overdraft, enabling withdrawal beyond balance up to a limit.

Transaction – Represents a record of each operation with details like type, amount, and timestamp.

AccountRepository – Handles storage and retrieval of account data using collections.

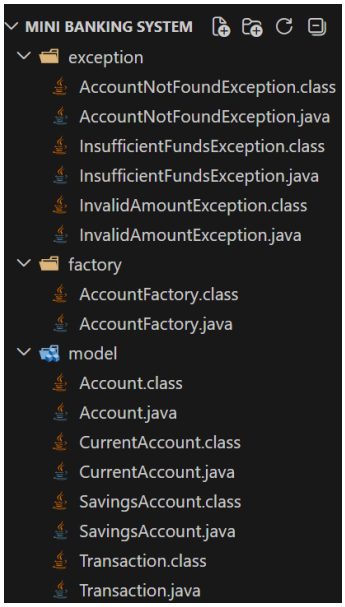
BankingService – Implements business logic for all operations such as create, deposit, withdraw, transfer, and balance check.

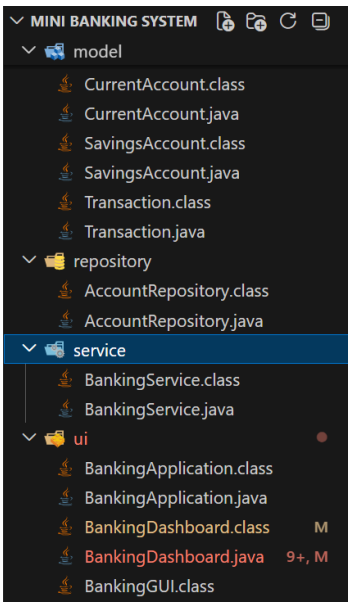
AccountFactory – Creates account objects based on type, centralizing object creation logic.

BankingGUI – Provides a graphical interface for user interaction through forms, buttons, and displays.

BankingApplication – Console-based interface enabling menu-driven banking operations.

PACKAGE STRUCTURE





INTERACTION FLOW

Step-by-Step Flow:

1) User Interaction (UI Layer)

- User enters input via:
  - CLI (BankingApplication)
  - GUI (BankingGUI)

Example:

- Enter account details
- Click "Deposit"

2) UI Calls Service Layer

- UI sends request to:

service.deposit(id, amount);

3) Service Layer Processes Logic

- BankingService:
  - Validates input
  - Applies business rules
  - Decides what operation to perform

4) Service Calls Repository

Account acc = repository.findById(id);

5) Repository Accesses Data

- Retrieves account from:

HashMap<Integer, Account>

6) Model Executes Operation

- Actual logic happens in:

acc.deposit(amount);
acc.withdraw(amount);

7) Transaction Recorded

- A Transaction object is created and stored

8) Response Back to UI

- Result returned → UI displays:
  - Success message
  - Error message

OOP CONCEPTS

Abstraction

Where it is used:

- Account abstract class

Why it is used:

- To define a common structure for all account types while hiding implementation details of withdrawal

Code Snippet:

```
public abstract class Account {

   protected int accountId;
   protected double balance;

   public void deposit(double amount) throws
InvalidAmountException {
      if(amount <= 0)
         throw new
InvalidAmountException("Invalid deposit
amount");

      balance += amount;
   }

   public abstract void withdraw(double amount)
      throws InsufficientFundsException,
InvalidAmountException;
}
```

Explanation:

- The class is declared abstract, so it cannot be instantiated.
- withdraw() is abstract, meaning subclasses must define their own logic.
- Common functionality like deposit() is already implemented.

Encapsulation

Where it is used:

- In Account and AccountRepository

Why it is used:

- To **protect data** and control access through methods

Code Snippet:

```
private Map<Integer, Account> accounts = new
HashMap<>();

public void save(Account account){
   accounts.put(account.getAccountId(), account);
}

public Account findById(int id){
   return accounts.get(id);
}
```

Explanation:

- The accounts map is private and cannot be accessed directly.
- Data is accessed and modified only through methods like save() and findById().

Inheritance

Where it is used:

- SavingsAccount and CurrentAccount extend Account

Why it is used:

- To **reuse common properties and methods** from the base class

Code Snippet:

```
public class SavingsAccount extends Account {

   public SavingsAccount(int id, String name,
double balance){
      super(id, name, balance);
   }
}
```

Explanation:

- SavingsAccount inherits all fields and methods from Account.
- super() is used to call the parent constructor.

Polymorphism

Where it is used:

- withdraw() method (overridden in subclasses)

Why it is used:

- To allow **different behaviors for the same method**

Code Snippet:

```
@Override
public void withdraw(double amount)
    throws InsufficientFundsException,
InvalidAmountException {

  if(balance < amount)
    throw new
InsufficientFundsException("Insufficient funds");

  balance -= amount;
}
```

Explanation:

- Same method withdraw() behaves differently in different classes.
- JVM decides which version to call at runtime.

Exception Handling

Where it is used:

- Custom exception classes and validations

Why it is used:

- To handle **invalid operations safely**

Code Snippet:

```
if(acc == null)
  throw new
AccountNotFoundException("Account not
found");

acc.deposit(amount);
```

Explanation:

- If account does not exist, an exception is thrown.
- Prevents program crash and provides meaningful error message.

Collections (Java List & Map)

Where it is used:

- List<Transaction> and Map<Integer, Account>

Why it is used:

- To store multiple objects efficiently in memory

Code Snippet:

```
protected List<Transaction> transactions = new
ArrayList<>();

transactions.add(new Transaction("DEPOSIT",
amount));
```

Explanation:

- ArrayList stores transaction history dynamically.
- Allows easy addition and retrieval of records.

IMPLEMENTATION HIGHLIGHTS

1. Deposit Logic with Validation + Transaction Recording

Code Snippet:

```
public void deposit(double amount) throws
InvalidAmountException {
  if(amount <= 0)
    throw new InvalidAmountException("Invalid
deposit amount");

  balance += amount;
  transactions.add(new Transaction("DEPOSIT",
amount));
}
```

Justification:

This ensures that **invalid inputs are prevented** and every valid transaction is recorded, which is essential for maintaining accurate account history.

Explanation:

- Checks if the amount is valid (greater than 0)
- Updates balance

- Creates a Transaction object and stores it
- Guarantees both **data correctness and traceability**

2. Different Withdrawal Logic (Savings vs Current)

Code Snippet:

```
if(balance + overdraftLimit < amount)
   throw new
InsufficientFundsException("Overdraft limit
exceeded");

balance -= amount;
transactions.add(new
Transaction("WITHDRAW", amount));
```

Justification:

Implements **real-world banking behavior** where current accounts allow overdraft, unlike savings accounts.

Explanation:

- Allows withdrawal beyond balance using overdraftLimit
- Prevents exceeding allowed limit
- Records transaction after successful withdrawal
- Makes account types behave differently

3. Fund Transfer Logic (Reuse of Methods)

Code Snippet:

```
public void transfer(int from,int to,double
amount) throws Exception {

   Account a = repository.findById(from);
   Account b = repository.findById(to);

   if(a==null || b==null)
      throw new
AccountNotFoundException("Account not
found");

   a.withdraw(amount);
   b.deposit(amount);
}
```

Justification:

Promotes **code reuse and consistency** by using existing deposit and withdraw methods instead of duplicating logic.

Explanation:

- Fetches both accounts
- Validates existence
- Uses withdraw() and deposit() instead of writing new logic
- Ensures all rules (like validation) are automatically applied

4. Factory-Based Object Creation

Code Snippet:

```
public static Account createAccount(String type,
int id, String name, double balance){

   if(type.equalsIgnoreCase("SAVINGS"))
      return new
SavingsAccount(id,name,balance);

   else if(type.equalsIgnoreCase("CURRENT"))
      return new
CurrentAccount(id,name,balance);

   else
      return null;
}
```

Justification:

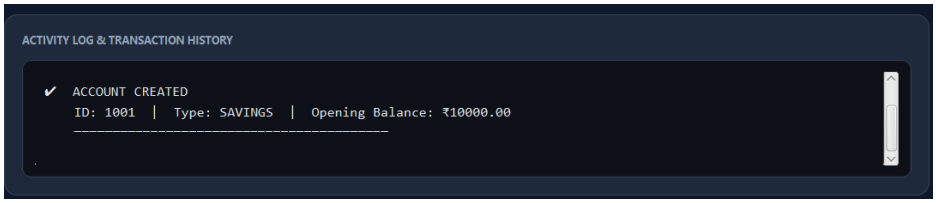
Centralizes object creation, making the system **flexible and easy to extend** when adding new account types.

Explanation:

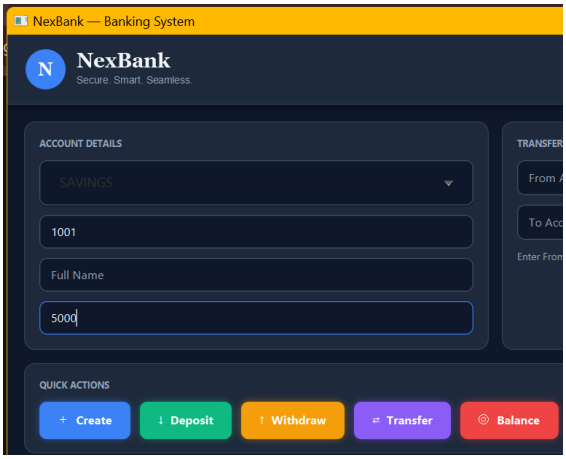
- Takes account type as input
- Returns appropriate object
- Removes object creation logic from UI/service
- Makes system cleaner and easier to maintain

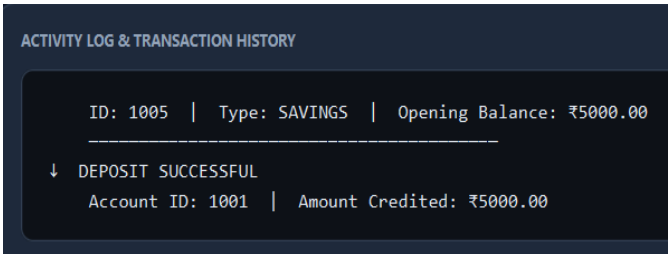
TESTING AND ERROR HANDLING

Account creation

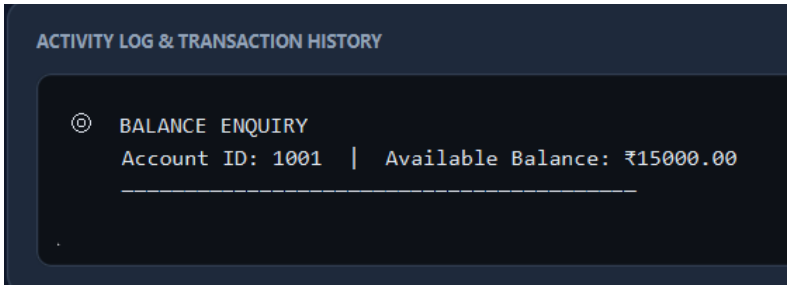


Deposit/Withdrawal

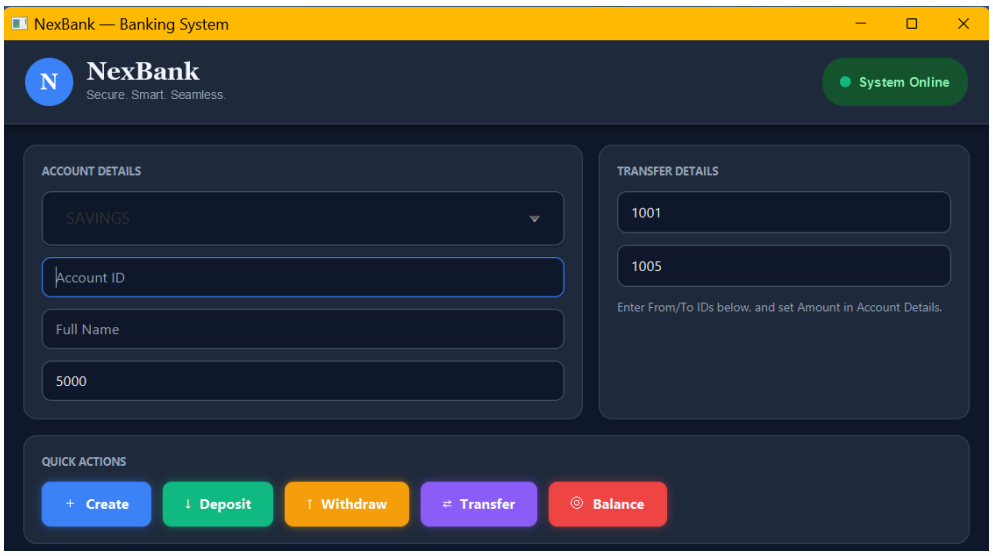


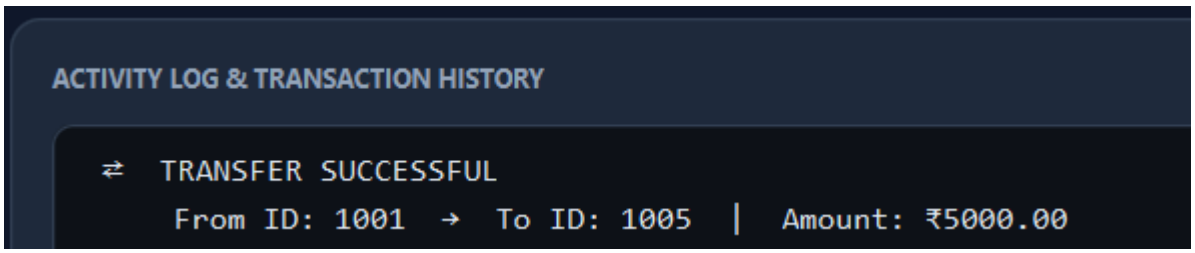


Check Balance

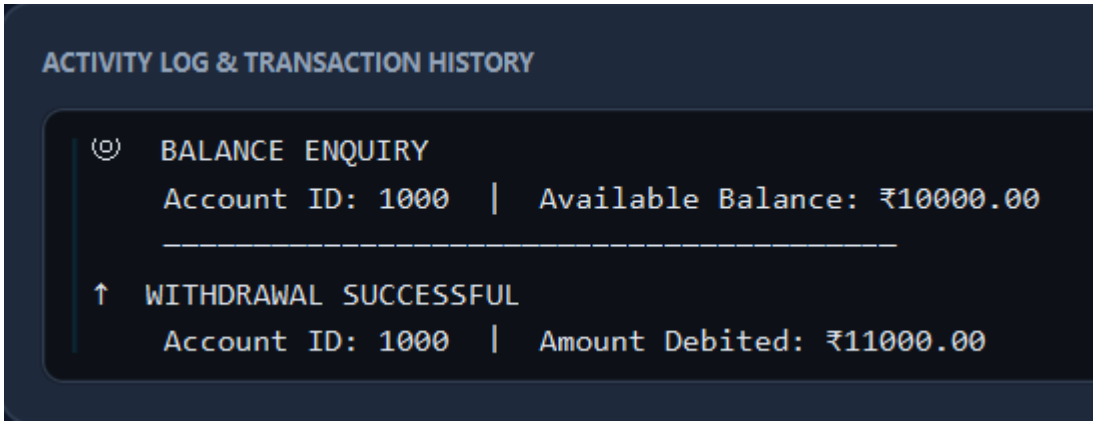


Transfer

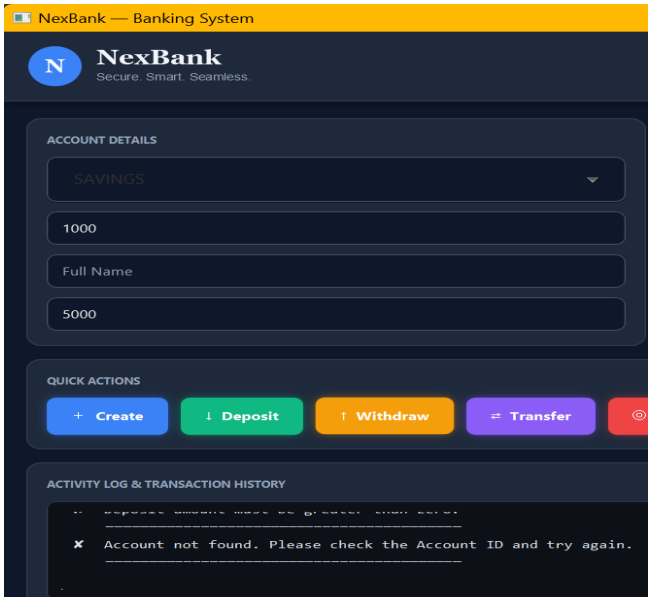


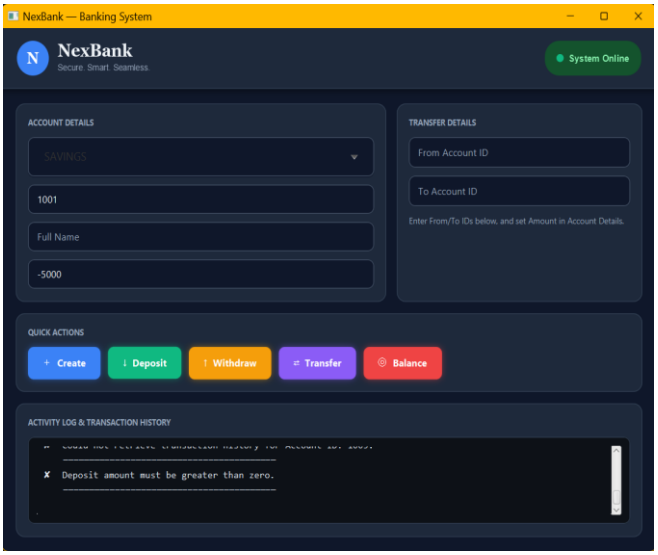


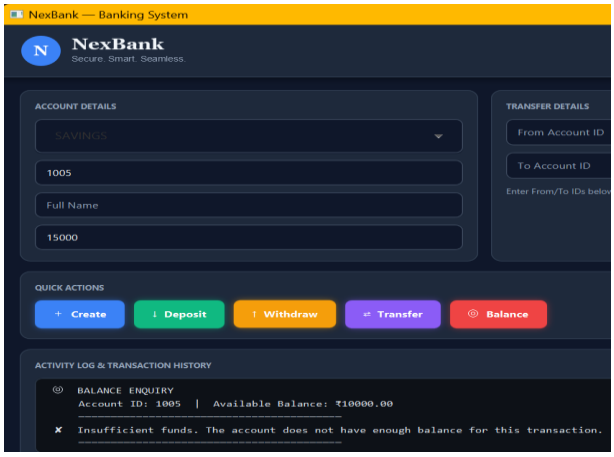
Current account overdraft



Edge cases







GIT WORKFLOW

```
commit 785ae826fcd2bee434c60b23d9c4b740f5ac60d1
Author: Prat06-creator <nandipratyusha6@gmail.com>
Date:   Mon Mar 23 00:34:51 2026 +0530

    implemented layered architecture and add banking UI

commit 69b92b42d21ebfdd9e360959c0fb712ada5d5d61
Merge: 02a9a96 1b42868
Author: Prat06-creator <nandipratyusha6@gmail.com>
Date:   Thu Mar 19 01:15:24 2026 +0530

    Merge branch 'main' of https://github.com/IEM-Gurukul/term-ii-project-submission-Prat06-creator

commit 02a9a96a852e601417364c1fd8fc84c8702db43e
Author: Prat06-creator <nandipratyusha6@gmail.com>
Date:   Thu Mar 19 01:11:33 2026 +0530

    Implemented account creation functionality in banking system
```

```
PS D:\term-ii-project-submission-Prat06-creator> git log
commit 5ae41a1213a83e4ca753c449bc04a933e0392e05 (HEAD -> main, origin/main)
Author: Prat06-creator <nandipratyusha6@gmail.com>
Date:   Fri Mar 27 11:04:42 2026 +0530

    Handled the edge cases for failure scenarios and displaying of proper error messages

commit ee587f65f69e3f5ce4ff5239c755c8605a4b395d
Author: Prat06-creator <nandipratyusha6@gmail.com>
Date:   Thu Mar 26 02:47:33 2026 +0530

    Removed unnecessary file i.e. BankingGUI.java

commit 8b6550cf93c257ffeefd5a69970ee0104aae20c9
Author: Prat06-creator <nandipratyusha6@gmail.com>
Date:   Thu Mar 26 02:41:40 2026 +0530

    Added proposal idea and overall idea of the project in README.md

commit f4230780fbe7a9ce5952d020fcec9c1c2d39e5b4
Author: Prat06-creator <nandipratyusha6@gmail.com>
Date:   Tue Mar 24 01:36:02 2026 +0530

    Implemented UI changes and created a file  BankingDashBoard.java

commit 785ae826fcd2bee434c60b23d9c4b740f5ac60d1
Author: Prat06-creator <nandipratyusha6@gmail.com>
Date:   Mon Mar 23 00:34:51 2026 +0530

    implemented layered architecture and add banking UI
```

CONCLUSION:

The Mini Banking System successfully demonstrates the implementation of a structured and modular application using layered architecture. By separating the system into UI, service, and repository layers, the project ensures better maintainability, scalability, and clarity of code. It effectively simulates core banking operations such as account creation, deposit, withdrawal, and fund transfer while maintaining data integrity through proper validation and exception handling. Additionally, the use of key OOP concepts like abstraction, encapsulation, inheritance, and polymorphism enhances code reusability and flexibility. Overall, the project provides a strong foundation for understanding real-world application design and object-oriented programming principles.

FUTURE SCOPE:

The Mini Banking System can be further enhanced by integrating a database (e.g., MySQL) for persistent data storage instead of in-memory collections. Security features like user authentication and encryption can be added to protect sensitive data. The system can also be expanded into a web or mobile application for better accessibility. Additional features such as loan management, interest calculation, and transaction analytics can be incorporated to make it more realistic and scalable.

TITLE: Mini Banking System using Layered Architecture (Service + Repository)

PROBLEM STATEMENT:

Modern banking software must be maintainable, scalable, and secure. Many beginner systems mix business logic, database access, and user interface code, making them difficult to maintain and extend. This project proposes a Mini Banking System implemented using Layered Architecture, separating responsibilities into presentation, service, and repository layers.

The system allows users to perform core banking operations such as creating accounts, depositing money, withdrawing money, checking balances, and viewing transaction history. By applying Object-Oriented Programming principles such as abstraction, inheritance, and polymorphism, the system becomes modular and reusable.

The layered architecture ensures separation of concerns, allowing independent modification of business logic and data storage mechanisms. This approach reflects real-world enterprise software design used in banking and financial systems.

TARGET USER / AUDIENCE : Bank customers performing basic banking operations, Bank staff managing customer accounts.

CORE FEATURES :

- Create new bank accounts
- Deposit money into account
- Withdraw money from account
- Check account balance
- View transaction history
- Transfer money between accounts
- Input validation and exception handling
- In-memory storage using Java Collections

OOPS CONCEPT USED:

1)Abstraction 2) Inheritance 3) Polymorphism 4) Encapsulation
5) Exception Handling 6) Collections 7) Design Pattern

BRIEF ARCHITECTURE DESCRIPTION :

The system follows a **layered object-oriented architecture** where an abstract Account class defines common properties and operations of bank accounts, and specialized subclasses such as SavingsAccount and CurrentAccount implement account-specific behavior. A central BankingService manages core banking operations like deposit, withdrawal, and fund transfer while enforcing business rules. The AccountRepository acts as the persistence layer responsible for storing, retrieving, and updating account data using Java Collections. This separation ensures clear boundaries between domain logic, business processing, and data management, improving maintainability and scalability.

Abstract Account class → Specialized Account subclasses → Banking Service → Repository (Persistence) layer

PCCCS495 Proposal Review – APPROVED_MINOR

S soumadip.biswas@ieminternational.ai

Reply Reply all Forward

To: Pratyusha Nandi

Tue 10/03/2026 10:11

Dear PRATYUSHA NANDI,

Project: Mini Banking System using Layered Architecture (Service + Repository)
Decision: APPROVED_MINOR
Score: 90

Strengths:
The proposal clearly identifies a common problem in beginner projects (mixed responsibilities) and proposes a standard, effective solution using a layered architecture. It explicitly lists key OOP principles (Abstraction, Inheritance, Polymorphism, Encapsulation) and other crucial Java concepts (Exception Handling, Collections) that will be applied. The features are well-defined and appropriate for a university-level project, offering a solid foundation for learning core software design principles. The use of in-memory storage via Java Collections is suitable for focusing on architectural and OOP concepts without adding database complexity. The separation of concerns between Service and Repository layers is well-articulated, reflecting good software engineering practices.

Improvements:
1. **User Interface (Presentation Layer):** While the problem statement mentions a 'presentation layer', the architecture description focuses heavily on the service and repository. The proposal should explicitly state *how* users will interact with the system (e.g., console-based UI, a simple Swing/JavaFX GUI, a basic web interface) to complete the layered architecture description.
2. **Specific Design Patterns:** The proposal lists 'Design Pattern' under OOP Concepts. It would be beneficial to specify *which* design patterns are intended to be implemented (e.g., Factory for account creation, Strategy for different account types' behaviors, Observer for transaction notifications) to demonstrate a deeper understanding and provide clearer learning goals.
3. **Testing Strategy:** The proposal does not mention any plans for testing (e.g., unit tests for service/repository layers, integration tests). Incorporating a basic testing strategy would significantly enhance the project's quality and demonstrate best practices.
4. **Error Handling Granularity:** While 'exception handling' is mentioned, providing more detail on specific types of exceptions (e.g., custom exceptions for business rule violations like InsufficientFundsException) and how they will be handled gracefully would improve robustness.
5. **Concurrency (Optional but valuable):** For a banking system, concurrent operations (e.g., multiple users accessing accounts simultaneously) are a real-world concern. While potentially advanced for an introductory project, considering or discussing how concurrent access to in-memory accounts would be managed (e.g., basic synchronization mechanisms) could be a valuable extension or discussion point.

Regards,
Course Instructor

Reply Forward